Système et Réseaux
CM5 : Compétition et synchronisation (2)

Pierre Rousselin
rousselin@univ-paris13.fr

Université Sorbonne Paris Nord
L2 informatique et double-licence

février 2026

Sections critiques et exclusion mutuelle

Deadlock et dîner des philosophes

Problème des producteurs et des consommateurs

Problème des lecteurs et des écrivains

Sections critiques et exclusion mutuelle

Deadlock et dîner des philosophes

Problème des producteurs et des consommateurs

Problème des lecteurs et des écrivains

Rappel `million.c`

```c
#define N 1000
void *bosser(void *);
int incremente_moi = 0;
int main(void)
{
    pthread_t th[N];
    int i;
    for (i = 0; i < N; ++i)
        pthread_create(&th[i], NULL, bosser, NULL);
    for (i = 0; i < N; ++i)
        pthread_join(th[i], NULL);

    printf("incremente_moi = %d\n", incremente_moi);
    return 0;
}
void *bosser(void *arg)
{
    int i;
    for (i = 0; i < N; ++i)
        ++incremente_moi;
    return NULL;
}
```

Problème de non-atomicité

- Le total ne fait pas toujours 1000000
- car l'instruction `++incremente_moi` n'est pas atomique (en assembleur sur ma machine, correspond en fait à 3 instructions).
- Il faudrait donc s'assurer que jamais plus d'un seul *thread* ne rentre dans cette portion de code.
- On parle de *section critique*.

Section critique

Une section critique est une partie du code (pas nécessairement contigüe) dans laquelle on doit assurer à un thread s'y trouvant qu'il est seul à accéder à certaines ressources.
Exemples :

- Dans `million.c` : l'incrémentation de `incremente_moi`.

Section critique

Une section critique est une partie du code (pas nécessairement contigüe) dans laquelle on doit assurer à un thread s'y trouvant qu'il est seul à accéder à certaines ressources.
Exemples :

- Dans `million.c` : l'incrémentation de `incremente_moi`.
- Autre exemple du cours précédent, une section critique en trois parties concernant l'accès en lecture/écriture à une variable partagée `n`.
  - Dans un premier thread :

    ```
    n = 3; /* section critique (1ère partie) */
    /* ... */
    n = 0; /* section critique (2ème partie) */
    ```

  - Dans un second thread :

    ```
    /* début section critique (3ème partie) */
    if (n != 0) {
        x = 8 / n;
    /* fin section critique (3ème partie) */
        /* ... */
    }
    ```

Section critique et sémaphore

Solution au problème de `million.c` (vue en TP) : utiliser un sémaphore binaire (c'est-à-dire à compteur 0 ou 1) :

```
sem_t verrou; /* exemple avec sémaphore global */
int main(void)
{
        pthread_t th[N];
        sem_init(&verrou, 0, 1); /* ouvert au départ */
        int i;
        for (i = 0; i < N; ++i)
                pthread_create(&th[i], NULL, bosser, NULL);
        for (i = 0; i < N; ++i)
                pthread_join(th[i], NULL);
        sem_destroy(&verrou);
        printf("incremente_moi = %d\n", incremente_moi);
}
void *bosser(void *arg)
{
        int i;
        for (i = 0; i < N; ++i) {
                sem_wait(&verrou);
                ++incremente_moi;
                sem_post(&verrou);
        }
        return NULL;
}
```

Protection d'une section critique par sémaphore

Lorsqu'on utilise un sémaphore pour protéger une ou plusieurs sections critiques :

- Le sémaphore est toujours initialisé à 1 car aucun *thread* n'a encore accédé à la section critique.
- Le sémaphore est toujours mis à 0 puis remis à 1 par le même *thread*.

Protection d'une section critique par sémaphore

Lorsqu'on utilise un sémaphore pour protéger une ou plusieurs sections critiques :

- Le sémaphore est toujours initialisé à 1 car aucun *thread* n'a encore accédé à la section critique.
- Le sémaphore est toujours mis à 0 puis remis à 1 par le même *thread*.

Cette situation étant très fréquente, une version spécialisée de ces sémaphores a été créée : les *mutex*.

million_mutex.c

```c
#define N 1000
void *bosser(void *);
int incremente_moi = 0;
pthread_mutex_t verrou; /* exple avec mutex global */
int main(void)
{
        pthread_t th[N];
        pthread_mutex_init(&verrou, NULL);
        for (int i = 0; i < N; ++i)
                pthread_create(&th[i], NULL, bosser, NULL);
        for (int i = 0; i < N; ++i)
                pthread_join(th[i], NULL);
        printf("incremente_moi = %d\n", incremente_moi);
        pthread_mutex_destroy(&verrou);
}
void *bosser(void *arg)
{
        int i;
        for (i = 0; i < N; ++i) {
                pthread_mutex_lock(&verrou);
                ++incremente_moi;
                pthread_mutex_unlock(&verrou);
        }
        return NULL;
}
```

4 opérations

MUTual EXclusion, exclusion mutuelle. On parle aussi de verrous.

```
#include <pthread.h>
pthread_mutex_init(pthread_mutex_t *m, NULL);
pthread_mutex_lock(pthread_mutex_t *m);
pthread_mutex_unlock(pthread_mutex_t *m);
pthread_mutex_destroy(pthread_mutex_t *m);
```

4 opérations

- Le type `pthread_mutex_t` est un type opaque, qui dépend des implémentations. Comme en POO, les détails sont cachés.
- `pthread_mutex_init(m, NULL)` initialise le *mutex* d'adresse `m` (au départ non verrouillé). Le deuxième argument de `pthread_mutex_init` concerne les attributs de *mutex*, on ne les utilise pas dans ce cours pour le moment et on met `NULL`.
- `pthread_mutex_lock(m)` est atomique et bloquant : si le *mutex* est déjà verrouillé, le thread est mis en sommeil.
- `pthread_mutex_unlock(m)` est atomique, déverrouille le *mutex* et réveille un et un seul des threads (s'il y en a) qui sont en attente sur le verrou.
- `pthread_mutex_destroy(m)` libère les ressources associées au *mutex*.

Résumé des appels système *mutex*/sémaphores

Sémaphores et *mutex* sont à mettre en parallèle.

```
#include <pthread.h>
int pthread_mutex_init(pthread_mutex_t *m, NULL);
int pthread_mutex_lock(pthread_mutex_t *m);
int pthread_mutex_unlock(pthread_mutex_t *m);
int pthread_mutex_destroy(pthread_mutex_t *m);
```

```
#include <semaphore.h>
int sem_init(sem_t *sem, 0, unsigned int valeur);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_destroy(sem_t *sem);
```

Remarquer la cohérence de l'API (toujours le *mutex* ou sémaphore comme premier paramètre, toujours via un pointeur).

Mutex vs Sémaphore

Un *mutex*

- devrait être verrouillé et déverrouillé par **le même *thread*** (dans le standard POSIX) ;

Mutex vs Sémaphore

Un *mutex*

- devrait être verrouillé et déverrouillé par **le même *thread*** (dans le standard POSIX) ;
- Donc à réserver aux sections critiques (mises à jour, test sur une variable, écriture dans un fichier, ...).

Un sémaphore

- peut tout faire (même jouer le rôle d'un *mutex*) ;
- mais c'est plus clair si on l'utilise dans tous les autres rôles, c'est-à-dire attendre/signaler la disponibilité d'une ressource ou la satisfaction d'une condition.

Où mettre un *mutex* ou un sémaphore ?

Les *threads* doivent connaître les adresses des *mutex* et/ou sémaphores qu'ils vont devoir manipuler. Ceux-ci peuvent être

- des variables globales (mais ne passe pas très bien à l'échelle) ;
- dans la pile du *thread* initial (à condition qu'il attende la fin de tous les autres) ;
- alloués dynamiquement sur le tas.

Dans les deux derniers cas, il faudra transmettre des adresses aux *threads*, d'une façon ou d'une autre... par exemple comme argument directement ou bien dans une structure dont l'adresse est transmise.

Exemple : *mutex* dans la pile de `main`

```
#define N 1000
void *bosser(void *);
int incremente_moi = 0;
int main(void)
{
    pthread_t th[N];
    pthread_mutex_t verrou;
    pthread_mutex_init(&verrou, NULL);
    for (int i = 0; i < N; ++i)
        pthread_create(&th[i], NULL, bosser, &verrou);
    for (int i = 0; i < N; ++i)
        pthread_join(th[i], NULL);
    printf("incremente_moi = %d\n", incremente_moi);
    pthread_mutex_destroy(&verrou);
}
void *bosser(void *arg) /* arg est en fait l'adresse d'un mutex */
{
    pthread_mutex_t *mut = arg;
    for (int i = 0; i < N; ++i) {
        pthread_mutex_lock(mut);
        ++incremente_moi;
        pthread_mutex_unlock(mut);
    }
    return NULL;
}
```

mutex sur le tas (et dans une structure), côté `main`

```
struct arg_bosser {
        pthread_mutex_t *verrou_incr_moi;
        int *incremente_moi;
};
void *bosser(void *ab); /* adresse d'une struct arg_bosser */
int main(void)
{
        pthread_t th[N];
        pthread_mutex_t *m = malloc(sizeof(pthread_mutex_t));
        int incremente_moi = 0; /* <- plus besoin d'être globale */
        pthread_mutex_init(m, NULL);
        struct arg_bosser *arg = malloc(sizeof(*arg));
        arg->verrou_incr_moi = m;
        arg->incremente_moi = &incremente_moi;
        for (int i = 0; i < N; ++i) {
                pthread_create(&th[i], NULL, bosser, arg);
        }
        for (int i = 0; i < N; ++i) {
                pthread_join(th[i], NULL);
        }
        printf("incremente_moi = %d\n", incremente_moi);
        pthread_mutex_destroy(m);
        free(m);
        free(arg);
}
```

mutex sur le tas (et dans une structure), côté `thread`

```
struct arg_bosser {
        pthread_mutex_t *verrou_incr_moi;
        int *incremente_moi;
};
void *bosser(void *arg)
{
        struct arg_bosser *ab = arg;
        int i;
        for (i = 0; i < N; ++i) {
                pthread_mutex_lock(ab->verrou_incr_moi);
                ++(ab->incremente_moi);
                pthread_mutex_unlock(ab->verrou_incr_moi);
        }
        return NULL;
}
```

Remarque : dans le cas où chaque *thread* a sa propre version de la structure allouée sur le tas, il peut être utile de la retourner pour pouvoir la libérer.

Section critique et atomicité

- En quelque sorte, une section critique est une partie du code qu'on aimerait atomique... mais qui ne l'est pas.
- Utiliser des mutex pour protéger l'accès à cette section remplit ce rôle.
- Les sections critiques doivent être les plus courtes possibles !
- Sinon on perd tout avantage à utiliser plusieurs *threads*.

```
void *bosser(void *arg)
{
    /* NE PAS FAIRE ÇA !!!! */
    /* CASSE LE PARALLÉLISME */
    pthread_mutex_lock(&m);
    for (int i = 0; i < N; ++i) {
        ++incremente_moi;
    }
    pthread_mutex_unlock(&m);
    return ab;
}
```

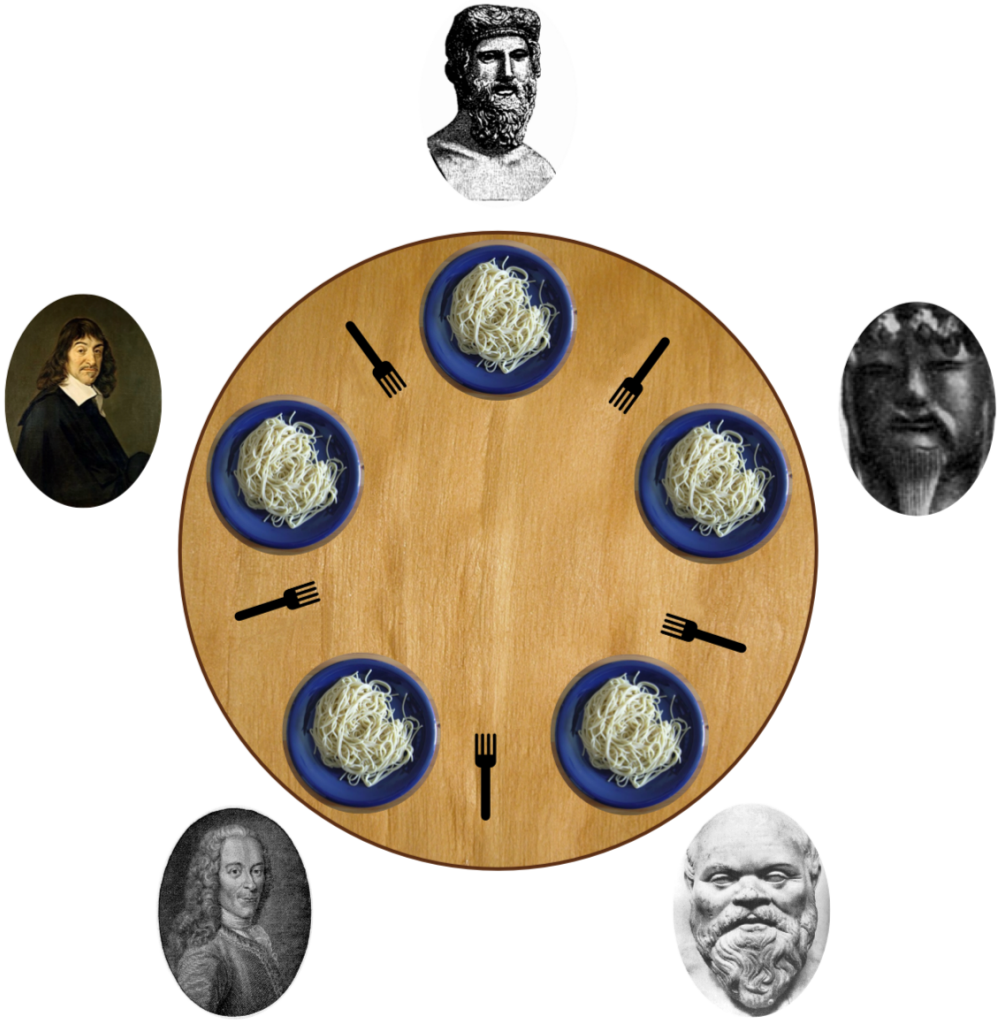
Sections critiques et exclusion mutuelle

Deadlock et dîner des philosophes

Problème des producteurs et des consommateurs

Problème des lecteurs et des écrivains

Un problème d'Edsger Dijkstra



Par Benjamin D. Esham / Wikimedia Commons, dans le sens des aiguilles d'une montre en partant de midi : Platon, Confucius, Socrate, Voltaire, Descartes.

Énoncé du problème

- 5 philosophes sont assis autour d'une table ronde ;
- chacun d'entre eux a en face de lui une assiette de spaghettis ;
- entre deux assiettes se trouve toujours une fourchette.
- Chaque philosophe réfléchit pendant un temps indéterminé
- ... puis a faim et essaie de manger.
- Pour manger il a besoin des fourchettes situées à sa gauche et à sa droite.
- S'il réussit à s'en emparer, il mange (pendant un certain temps) puis repose les fourchettes.

philosophes.c (1)

```
void *philosophe(void *arg);
void reflechir(int i);
void manger(int i);
pthread_mutex_t fourchettes[5];
int main(void)
{
    pthread_t th[5];
    int i, nums[] = {0, 1, 2, 3, 4};
    for (i = 0; i < 5; ++i)
        pthread_mutex_init(&fourchettes[i], NULL);
    for (i = 0; i < 5; ++i)
        pthread_create(&th[i], NULL, philosophe, &nums[i]);
    for (i = 0; i < 5; ++i)
        pthread_join(th[i], NULL);
    for (i = 0; i < 5; ++i)
        pthread_mutex_destroy(&fourchettes[i]);
}
```

philosophes.c (2)

```c
void *philosophe(void *arg)
{
    int i = *((int *) arg);
    for (;;) {
        reflechir(i);
        pthread_mutex_lock(&fourchettes[i]);
        printf("Le philosophe %d a pris sa fourchette gauche.\n", i);
        pthread_mutex_lock(&fourchettes[(i + 1) % 5]);
        printf("Le philosophe %d a pris sa fourchette droite.\n", i);
        manger(i);
        pthread_mutex_unlock(&fourchettes[i]);
        printf("Le philosophe %d a posé sa fourchette gauche.\n", i);
        pthread_mutex_unlock(&fourchettes[(i + 1) % 5]);
        printf("Le philosophe %d a posé sa fourchette droite.\n", i);
    }
}
void reflechir(int i) { printf("Le philosophe %d réfléchit.\n", i); }
void manger(int i) { printf("Le philosophe %d mange.\n", i); }
```

Interblocage

Au bout d'un certain temps...

```
Le philosophe 2 a pris sa fourchette gauche.
Le philosophe 1 réfléchit.
Le philosophe 0 mange.
Le philosophe 0 a posé sa fourchette gauche.
Le philosophe 3 a pris sa fourchette gauche.
Le philosophe 1 a pris sa fourchette gauche.
Le philosophe 0 a posé sa fourchette droite.
Le philosophe 0 réfléchit.
Le philosophe 0 a pris sa fourchette gauche.
Le philosophe 4 a pris sa fourchette gauche.
```

Interblocage

Au bout d'un certain temps...

```
Le philosophe 2 a pris sa fourchette gauche.
Le philosophe 1 réfléchit.
Le philosophe 0 mange.
Le philosophe 0 a posé sa fourchette gauche.
Le philosophe 3 a pris sa fourchette gauche.
Le philosophe 1 a pris sa fourchette gauche.
Le philosophe 0 a posé sa fourchette droite.
Le philosophe 0 réfléchit.
Le philosophe 0 a pris sa fourchette gauche.
Le philosophe 4 a pris sa fourchette gauche.
```

Tous les philosophes ont pris leur fourchette gauche ! Et ils sont tous en attente sur leur fourchette de droite. Le programme est bloqué.

Deadlock

Un *deadlock*, (aussi appelé *interblocage* ou encore *étreinte mortelle*) est une situation où différents acteurs (*threads*, processus, personnes, administrations, ...) s'attendent les uns les autres.
Exemples :

- Les philosophes sont en *deadlock* circulaire au bout d'un moment.
- Pour avoir le laisser-passer A38, il faut remplir le formulaire jaune, pour obtenir le formulaire jaune, il faut remplir le formulaire bleu et pour obtenir le formulaire bleu il faut le laisser-passer A38.

Deadlock

Un *deadlock*, (aussi appelé *interblocage* ou encore *étreinte mortelle*) est une situation où différents acteurs (*threads*, processus, personnes, administrations, ...) s'attendent les uns les autres.
Exemples :

- Les philosophes sont en *deadlock* circulaire au bout d'un moment.
- Pour avoir le laisser-passer A38, il faut remplir le formulaire jaune, pour obtenir le formulaire jaune, il faut remplir le formulaire bleu et pour obtenir le formulaire bleu il faut le laisser-passer A38.

Un *deadlock* est un *bug* gravissime : les *threads* sont totalement bloqués et ne peuvent être libérés que par une intervention extérieure.
Éviter les *deadlocks* doit être une priorité.

Sections critiques et exclusion mutuelle

Deadlock et dîner des philosophes

Problème des producteurs et des consommateurs

Problème des lecteurs et des écrivains

Le problème des producteurs et des consommateurs

Problème énoncé (encore) par Edsger W. Dijkstra en 1965.
On considère :

- des producteurs ;
- des consommateurs ;
- un hangar ayant N places.
- Le consommateur veut prendre un produit dans le hangar puis le consommer ;
- le producteur produit et veut mettre ce produit dans le hangar.

Ce qui peut poser problème

- Le producteur dépose un produit dans le hangar mais celui-ci est déjà plein ;
- le consommateur prend un produit qui n'existe pas dans le hangar ;

Ce qui peut poser problème

- Le producteur dépose un produit dans le hangar mais celui-ci est déjà plein ;
- le consommateur prend un produit qui n'existe pas dans le hangar ;
- non-atomicité : deux consommateurs prennent tous les deux le même produit ;
- non-atomicité : deux producteurs posent leur produit à la même place.

Exemple avec une pile bornée

```
#define ELEM char
#define HAUTEUR_MAX_PILE 10
ELEM pile[HAUTEUR_MAX_PILE];
int hauteur_pile = 0;
ELEM produire(void);
void utiliser(ELEM produit);
void *producteur(void *arg);
void *consommateur(void *arg);
int main(void) { /* ... */ }
void *producteur(void *arg)
{
    ELEM produit = produire();
    pile[hauteur_pile++] = produit;
}
void *consommateur(void *arg)
{
    ELEM produit = pile[--hauteur_pile];
    utiliser(produit);
}
```

Problèmes

- Segmentation fault car lecture ou écriture en-dehors de `pile`.
- Même si `pile` n'était pas bornée, le consommateur risque de dépiler la pile vide.
- Non atomicité de l'accès à la pile. Exemple :
  - Un consommateur va consommer, il lit `hauteur_pile` qui vaut 1, ce 1 est stocké dans un registre.
  - Commutation de contexte, un autre consommateur lit `hauteur_pile` qui vaut toujours 1, consomme (lit) `pile[0]` puis met `hauteur_pile` à `0`.
  - Commutation de contexte, retour au premier consommateur. Le registre est restauré à 1, `pile[0]` est consommé une deuxième fois.

Section critique $\Longrightarrow$ Mutex

```
pthread_mutex_t maj_pile;
int main(void)
{
    pthread_mutex_init(&maj_pile, NULL);
    /* ... */
    pthread_mutex_destroy(&maj_pile);
    /* ... */
}
void *producteur(void *arg)
{
    ELEM produit = produire();
    pthread_mutex_lock(&maj_pile);
    pile[hauteur_pile++] = produit;
    pthread_mutex_unlock(&maj_pile);
}
void *consommateur(void *arg)
{
    pthread_mutex_lock(&maj_pile);
    ELEM produit = pile[--hauteur_pile];
    pthread_mutex_unlock(&maj_pile);
    utiliser(produit);
}
```

Deux ressources

- les places dans l'entrepôt ;
- les produits dans l'entrepôt ;
- un producteur produit un objet mais consomme une place ;
- un consommateur produit une place mais consomme un objet.

Si on suppose qu'initialement l'entrepôt est vide.

```
sem_t choses_sur_pile, place_dans_pile;
int main(void) {
    sem_init(&choses_sur_pile, 0, 0);
    sem_init(&place_dans_pile, 0, TAILLE_MAX_PILE);
    /* ... */
    sem_destroy(&choses_sur_pile);
    sem_destroy(&place_dans_pile);
    /* ... */
}
```

Producteurs et consommateurs : solution

```
sem_init(&choses_sur_pile, 0, 0);
sem_init(&place_dans_pile, 0, TAILLE_MAX_PILE);
void *producteur(void *arg)
{
    ELEM produit = produire();
    sem_wait(&place_dans_pile);
    pthread_mutex_lock(&maj_pile);
    pile[hauteur_pile++] = produit;
    pthread_mutex_unlock(&maj_pile);
    sem_post(&choses_sur_pile);
}
void *consommateur(void *arg)
{
    sem_wait(&choses_sur_pile);
    pthread_mutex_lock(&maj_pile);
    ELEM produit = pile[--hauteur_pile];
    pthread_mutex_unlock(&maj_pile);
    sem_post(&place_dans_pile);
    utiliser(produit);
}
```

Sections critiques et exclusion mutuelle

Deadlock et dîner des philosophes

Problème des producteurs et des consommateurs

Problème des lecteurs et des écrivains

Le problème des lecteurs et des écrivains

Le problème :

- des lecteurs ;
- des écrivains ;
- une bibliothèque.

- Si un écrivain est dans la bibliothèque, celle-ci doit être fermée à toute autre personne ($\approx$ mise à jour de la base de données) ;
- si un lecteur est dans la bibliothèque, d'autres lecteurs peuvent y entrer, mais pas d'écrivain ($\approx$ consultation de la base de donnée).

Le problème des lecteurs et des écrivains

Le problème :

- des lecteurs ;
- des écrivains ;
- une bibliothèque.

- Si un écrivain est dans la bibliothèque, celle-ci doit être fermée à toute autre personne ($\approx$ mise à jour de la base de données) ;
- si un lecteur est dans la bibliothèque, d'autres lecteurs peuvent y entrer, mais pas d'écrivain ($\approx$ consultation de la base de donnée).

Modélise des situations courantes d'accès/modifications dans des bases de données. Encore dû à Edsger Dijkstra.

Analogie

- Lorsqu'un utilisateur (lecteur ou écrivain) entre dans la bibliothèque, il allume la lumière.
- Un écrivain peut donc entrer si et seulement si la lumière est éteinte.
- Lorsqu'un lecteur entre dans la bibliothèque, il fait un bâton avec une craie sur la porte ;
- lorsqu'il en sort, il efface un bâton, s'il n'y a plus de bâton, il faut éteindre la lumière !
- Si un lecteur voit la bibliothèque allumée mais pas de bâton : il y a un écrivain, il faut attendre qu'elle s'éteigne ;
- sinon il y a des lecteurs, on peut entrer.

Les bâtons et la lumière

Par exemple, variable globale.

```
int num_lect_bib = 0;
```

Pour manipuler cette variable... réflexe...

Les bâtons et la lumière

Par exemple, variable globale.

```
int num_lect_bib = 0;
```

Pour manipuler cette variable... réflexe...

```
pthread_mutex_t update_num_lects_bib;
```

Les bâtons et la lumière

Par exemple, variable globale.

```
int num_lect_bib = 0;
```

Pour manipuler cette variable... réflexe...

```
pthread_mutex_t update_num_lects_bib;
```

La condition d'entrée pour les écrivains et le premier lecteur : que la lumière soit éteinte (que la bibliothèque soit vide).

Les bâtons et la lumière

Par exemple, variable globale.

```
int num_lect_bib = 0;
```

Pour manipuler cette variable... réflexe...

```
pthread_mutex_t update_num_lects_bib;
```

La condition d'entrée pour les écrivains et le premier lecteur : que la lumière soit éteinte (que la bibliothèque soit vide).

```
sem_t bib_est_vide;
```

On suppose la bibliothèque vide au départ :

```
sem_init(&bib_est_vide, 0, 1);
```

Solution pour les écrivains

Solution pour les écrivains

```
sem_wait(&bib_est_vide);
ecrire();
sem_post(&bib_est_vide);
```

Solution pour les lecteurs

Solution pour les lecteurs

```
pthread_mutex_lock(&update_num_lects_bib);
if (num_lect_bib == 0)
    sem_wait(&bib_est_vide);
++num_lect_bib;
pthread_mutex_unlock(&update_num_lects_bib);

lire();

pthread_mutex_lock(&update_num_lects_bib);
--num_lect_bib;
if (num_lect_bib == 0)
    sem_post(&bib_est_vide);
pthread_mutex_unlock(&update_num_lects_bib);
```

Famine de l'écrivain

Avec notre solution, que se passe-t-il si :

- Le premier lecteur arrive à $t = 0$ et lit pendant 2 unités de temps ;
- un écrivain arrive à $t = 1$;
- un deuxième lecteur arrive à $t = 1$ et lit pendant 2 unités de temps ;
- un troisième lecteur arrive à $t = 2$ et lit pendant 2 unités de temps ;
- etc

Famine de l'écrivain

Avec notre solution, que se passe-t-il si :

- Le premier lecteur arrive à $t = 0$ et lit pendant 2 unités de temps ;
- un écrivain arrive à $t = 1$;
- un deuxième lecteur arrive à $t = 1$ et lit pendant 2 unités de temps ;
- un troisième lecteur arrive à $t = 2$ et lit pendant 2 unités de temps ;
- etc

On parle de famine (*starvation*) lorsque, dans certains contextes, une tâche peut être en attente pour toujours.

Famine de l'écrivain

Avec notre solution, que se passe-t-il si :

- Le premier lecteur arrive à $t = 0$ et lit pendant 2 unités de temps ;
- un écrivain arrive à $t = 1$;
- un deuxième lecteur arrive à $t = 1$ et lit pendant 2 unités de temps ;
- un troisième lecteur arrive à $t = 2$ et lit pendant 2 unités de temps ;
- etc

On parle de famine (*starvation*) lorsque, dans certains contextes, une tâche peut être en attente pour toujours. Avez-vous des idées de solutions pour éviter une telle famine des écrivains ?